LEARNING GUIDE · MINI PROJECT AI

Knight Trap

Implementasi AI pada Game Knight Trap Menggunakan Algoritma Minimax dengan Alpha-Beta Pruning

Clarrence Adriano Hemeldan 0706012410066

Bagianmu: Antarmuka (UI), BFS & wilayah (territory), uji win rate dan waktu respons

Artificial Intelligence · Information Management & Technology · Universitas Ciputra 2026/2027

Disusun dari proposal (v1.1) dan kode di repo KnightTrap · 3 November 2026

Daftar Isi

1. Peta Belajarmu
2. Gambaran Proyek (Proposal Bab 1)
3. Aturan Game → Kode (Proposal Bab 2.1)
4. Alur Satu Giliran di Kode
5. Konsep AI yang Wajib Dipahami Semua Anggota
6. Antarmuka: Bagaimana Papan Digambar
7. BFS: Breadth-First Search
8. Wilayah (Territory) dan Skor Evaluasi
9. Pemain Pembanding (Baseline)
10. Pengujian Win Rate dan Waktu Respons
11. Hasil Pengujian (Proposal Bab 2.2)
12. Tambahan di Luar Proposal: Makan dan Waktu Giliran
13. Latihan
14. Pertanyaan Presentasi yang Mungkin Muncul
15. Checklist Sebelum Presentasi

1 Peta Belajarmu

Setelah mempelajari guide ini, kamu bisa:

- Menjelaskan bagaimana papan, knight, kotak terbakar, jebakan, dan penanda lompatan legal digambar.
- Menjelaskan BFS langkah demi langkah dan menghitung jarak knight dengan tangan.
- Menjelaskan cara wilayah (territory) dihitung dari dua hasil BFS, termasuk aturan seri.
- Menjelaskan pemain pembanding (acak dan greedy) serta cara win rate dan waktu respons diukur.

Tugasmu di Timeline (Bab 2.3)

Minggu	Pekerjaan	Kode
1–2	Antarmuka: papan, knight, kotak terbakar/jebakan, penanda lompatan legal, penunjuk giliran, pesan menang/kalah	`board.js`, `css/style.css`, `index.html`, `main.js` (`ui`)
2–3	BFS: jumlah lompatan ke setiap kotak, wilayah, skor evaluasi	`ai.js`: `bfs()`, bagian wilayah di `evaluate()`
3	Penyatuan BFS + DFS ke evaluasi (bersama Shatrya)	`ai.js`: `evaluate()`
3	Simulasi win rate vs acak & greedy, waktu respons per level	`tests/run.mjs` bagian 3–4; `randomMove()`, `greedyMove()`

2 Gambaran Proyek (Proposal Bab 1)

Apa yang kita buat

Knight Trap adalah varian dari permainan papan *Isolation* (1972). Dua knight ada di papan 7 × 7. Pada gilirannya, pemain:

1. **Lompat**: menggerakkan knight seperti kuda catur (huruf L) ke kotak yang belum terbakar atau dijebak.
2. **Bakar**: kotak yang ditinggalkan langsung terbakar dan tidak bisa dipakai lagi.
3. **Jebak**: menghancurkan satu kotak terbuka lain di mana pun.

Pemain yang **tidak punya lompatan legal** pada gilirannya kalah. Di versi kode, mendarat tepat di kotak knight musuh juga langsung menang (**makan**, tambahan di luar proposal; lihat bab "Tambahan di Luar Proposal"). Game ini dibuat untuk **rally games** (NPLC Universitas Ciputra, tema *Human vs AI*), jadi aturannya harus bisa dijelaskan dalam satu menit, satu game selesai dalam beberapa menit, dan lawan AI-nya tidak boleh terlalu mudah.

Masalah yang diselesaikan (Bab 1.2)

Kita harus membuat lawan AI yang (1) memilih langkah yang kuat, (2) cukup cepat untuk browser, dan (3) punya beberapa tingkat kesulitan. Ada dua tantangan utama:

- **Pohon permainan besar.** Satu langkah = pilihan lompatan (maks. 8) × pilihan kotak jebakan (bisa puluhan). Jumlah posisi meledak setiap giliran.
- **Menilai posisi tanpa main sampai habis.** Kita butuh *fungsi evaluasi* yang memberi skor pada posisi di tengah permainan.

Solusinya, sesuai ketentuan tugas yang membatasi kita ke **empat algoritma**: **Minimax** + **Alpha-Beta Pruning** untuk mengambil keputusan, dan **BFS** + **DFS** sebagai dasar fungsi evaluasi.

Peta file kode

File	Isi	Pemilik utama (Timeline)
`js/config.js`	Konstanta: ukuran papan, posisi awal, kedalaman tiap level, lama satu giliran, batas AI	Shatrya (setup), Louis (AI)
`js/rules.js`	Aturan game: lompatan legal, bakar, jebakan, makan, cek kalah. Tidak menyentuh DOM	Shatrya
`js/ai.js`	Minimax + Alpha-Beta, BFS, DFS, fungsi evaluasi, pemain acak & greedy	Louis (search), Clarrence (BFS), Shatrya (DFS)
`js/ai.worker.js`, `js/ai-client.js`	Menjalankan AI di Web Worker supaya tampilan tidak macet	Louis
`js/game.js`	Alur giliran: input pemain, giliran AI, penghitung waktu giliran, game over	Shatrya & Louis
`js/board.js`, `css/style.css`, `index.html`	Tampilan papan, knight, penanda lompatan, layout	Clarrence
`js/main.js`, `js/i18n.js`	Layar menu, pengaturan, Cara Bermain, panel statistik, teks dua bahasa	Clarrence & Louis
`js/net.js`	Multiplayer online (PeerJS)	Tambahan di luar proposal
`tests/run.mjs`	Semua pengujian Bab 2.2, plus uji makan dan waktu giliran	Louis, Clarrence (pengujian)

> **Repo dengan komentar per baris.** Folder `KnightTrap-Annotated` berisi kode yang sama persis, tetapi setiap baris diberi komentar bahasa Indonesia. Pakai repo itu untuk belajar; potongan kode di guide ini diambil dari repo asli `KnightTrap` supaya nomor barisnya cocok dengan kode yang dijalankan.

3 Aturan Game → Kode (Proposal Bab 2.1)

Setiap aturan di proposal punya tempat yang jelas di `js/rules.js`. File ini adalah **satu-satunya sumber aturan**: tampilan, AI, dan mode online semuanya memakai fungsi yang sama.

Aturan proposal	Kode
Papan 7 × 7, array 49 kotak (kosong / terbakar / jebakan) + posisi 2 knight	`createState()`, konstanta `EMPTY`, `BURNED`, `TRAP`
(a) Knight bergerak huruf L	`JUMPS` dan `NEIGHBORS` (dihitung sekali di awal)
(b) Kotak yang ditinggalkan terbakar	`applyJump()`
(c) Hancurkan satu kotak terbuka	`applyTrap()`; validasi di `validateMove()`
(d) Tidak boleh mendarat di kotak terbakar atau jebakan. Proposal juga melarang kotak lawan; di kode, mendarat di sana = makan (tambahan)	`legalJumps()` (cek `cells[n] === EMPTY`); `isOpen()` untuk kotak jebakan
(e) Tidak punya lompatan legal = kalah	`winner()` memakai `canMove()`
Tambahan: dimakan = kalah, langkah makan tanpa jebakan	`winner()` (`pos[0] === pos[1]`), `validateMove()`

Representasi papan

Kotak diberi nomor 0 sampai 48: `index = baris × 7 + kolom`, baris 0 adalah baris paling atas (rank 7). Jadi **a1 = 42** (kiri bawah) dan **g7 = 6** (kanan atas).

js/config.js : baris 3–8

```
3  export const N = 7;
4  export const SIZE = N * N;
5
6  // The proposal does not fix start squares. P1 starts at a1 (bottom-left),
7  // P2 at g7 (top-right). Index = row * 7 + col, row 0 is the top rank (7).
8  export const START = Object.freeze({ p1: 42, p2: 6, first: 0 });
```

js/rules.js : baris 28–35

```
28 export function createState(start = START) {
29   return {
30     cells: new Int8Array(SIZE),
31     pos: [start.p1, start.p2],
32     turn: start.first ?? 0,
33     moves: 0,
34   };
35 }
```

`cells` memakai `Int8Array` (array angka 8-bit) karena kecil dan cepat disalin. `pos[0]` adalah posisi knight P1, `pos[1]` knight P2, `turn` adalah siapa yang jalan (0 atau 1).

Tetangga knight dihitung sekali

js/rules.js : baris 10–26

```
const JUMPS = [
  [-2, -1], [-2, 1], [-1, -2], [-1, 2],
  [1, -2], [1, 2], [2, -1], [2, 1],
];

// Knight neighbor lists, computed once.
export const NEIGHBORS = Array.from({ length: SIZE }, (_, i) => {
  const r = Math.floor(i / N);
  const c = i % N;
  const out = [];
  for (const [dr, dc] of JUMPS) {
    const rr = r + dr;
    const cc = c + dc;
    if (rr >= 0 && rr < N && cc >= 0 && cc < N) out.push(rr * N + cc);
  }
  return out;
});
```

Untuk setiap kotak, kita coba 8 arah L dan simpan hanya yang masih di dalam papan. Hasilnya tabel 49 daftar. AI memanggil daftar ini jutaan kali, jadi menghitungnya sekali di awal menghemat banyak waktu.

Satu giliran = lompat, bakar, jebak

js/rules.js : baris 54–56

```
export function isOpen(s, sq) {
  return s.cells[sq] === EMPTY && sq !== s.pos[0] && sq !== s.pos[1];
}
```

js/rules.js : baris 60–64

```
export function legalJumps(s, side = s.turn) {
  const out = [];
  for (const n of NEIGHBORS[s.pos[side]]) if (s.cells[n] === EMPTY) out.push(n);
  return out;
}
```

js/rules.js : baris 77–83

```
export function applyJump(s, to) {
  const side = s.turn;
  const from = s.pos[side];
  s.cells[from] = BURNED;
  s.pos[side] = to;
  return from;
}
```

js/rules.js : baris 86–88

```
export function applyTrap(s, sq) {
  if (sq >= 0) s.cells[sq] = TRAP;
}
```

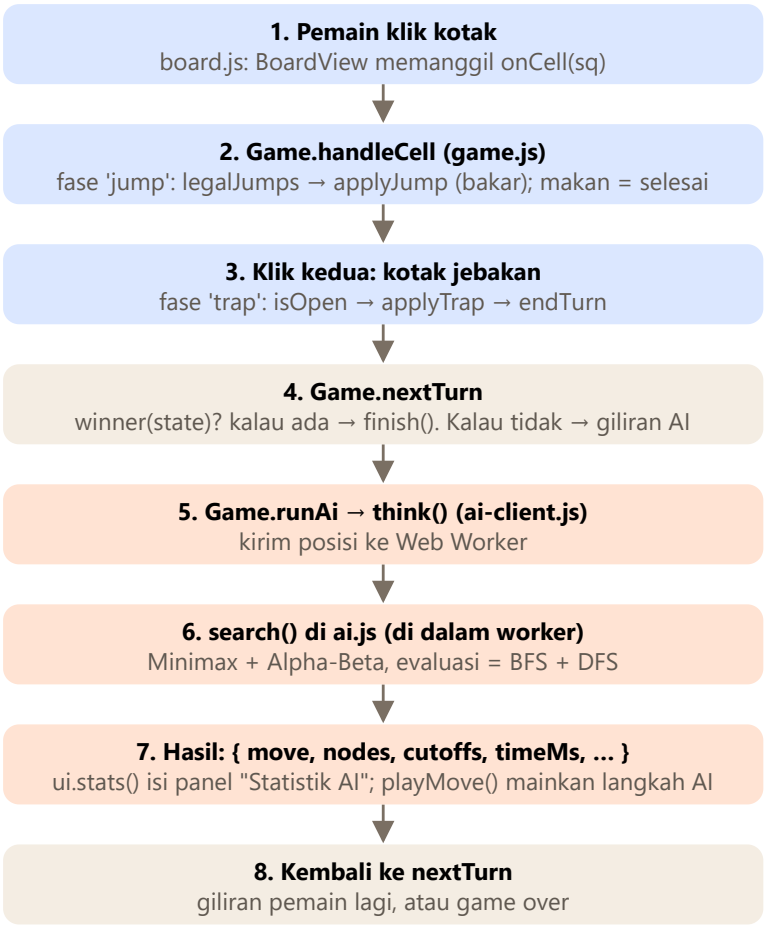
js/rules.js : baris 116–119

```
116  export function winner(s) {
117    if (s.pos[0] === s.pos[1]) return s.turn ^ 1;
118    return canMove(s, s.turn) ? null : s.turn ^ 1;
119  }
```

Kenapa `winner()` mengembalikan `s.turn ^ 1`? `^` adalah XOR. `0 ^ 1 = 1` dan `1 ^ 1 = 0`, jadi ini cara singkat menulis "pemain lainnya". Kalau yang sedang jalan tidak bisa bergerak, atau baru saja dimakan (kedua knight di kotak yang sama), lawannya menang.

4 Alur Satu Giliran di Kode

Bagian ini menghubungkan semua file. Setiap anggota harus bisa menceritakan alur ini saat presentasi.



Gambar: alur satu ronde (pemain lalu AI). Biru = input pemain, krem = alur game, oranye = AI.

js/game.js : baris 127–146

```
127   nextTurn() {
128     if (this.disposed) return;
129     const w = winner(this.state);
130     if (w !== null) return this.finish(w);
131     const p = this.current;
132     if (p.kind === 'human') {
133       this.phase = 'jump';
134       this.startClock();
135       this.refresh();
136     } else if (p.kind === 'ai') {
137       this.phase = 'ai';
138       this.stopClock();
139       this.refresh();
140       this.runAi();
141     } else {
142       this.phase = 'remote';
143       this.startClock();
144       this.refresh();
145     }
146   }
```

js/game.js : baris 209–233

```
  async runAi() {
    this.ui.thinking(true);
    const t0 = performance.now();
    let res = null;
    try {
      res = await think(this.state, { depth: this.ai.depth, prune: this.ai.prune });
    } catch (err) {
      console.error('AI failed', err);
    }
    if (this.disposed) return;
    const elapsed = performance.now() - t0;
    if (elapsed < AICFG.minUiDelayMs) await sleep(AICFG.minUiDelayMs - elapsed);
    if (this.disposed) return;
    this.ui.thinking(false);
    let move = res?.move;
    if (!move || !validateMove(this.state, move)) {
      console.warn('AI returned no valid move; falling back to a random legal move');
      move = randomMove(this.state);
    } else {
      this.ui.stats(res);
    }
    await this.playMove(move);
    if (this.disposed) return;
    this.nextTurn();
  }
```

Perhatikan dua pengaman di `runAi`: (1) kalau AI terlalu cepat (Easy), tampilan menunggu minimal 450 ms supaya tidak terasa "langsung"; (2) kalau AI gagal atau mengembalikan langkah tidak sah, game memakai langkah acak yang legal, jadi game tidak pernah macet.

js/ai-client.js : baris 44–53

```
export function think(state, { depth, prune }) {
  const args = { state: serialize(state), depth, prune };
  const w = getWorker();
  if (!w) return runInline(args);
  return new Promise((resolve, reject) => {
    const id = ++seq;
    pending.set(id, { resolve, reject, args });
    w.postMessage({ id, ...args });
  });
}
```

js/ai.worker.js : baris 5–13

```
self.onmessage = (e) => {
  const { id, state, depth, prune } = e.data;
  try {
    const result = search(deserialize(state), { depth, prune });
    self.postMessage({ id, result });
  } catch (err) {
    self.postMessage({ id, error: String(err && err.message ? err.message : err) });
  }
};
```

Kenapa Web Worker? JavaScript di browser berjalan di satu thread. Kalau Expert berpikir 1–2 detik di thread utama, tombol dan penghitung waktu akan membeku. Worker menjalankan `search()` di thread terpisah, jadi papan tetap hidup saat AI berpikir.

5 Konsep AI yang Wajib Dipahami Semua Anggota

Dosen bisa bertanya ke siapa saja. Bagian ini adalah dasar yang harus dikuasai semua anggota, walaupun detailnya milik anggota lain.

Agen dan model P.E.A.S (Tabel 2.1.1)

Komponen	Di Knight Trap	Di kode
Performance	Win rate AI, waktu berpikir ≤ 4 detik, jumlah posisi diperiksa sedikit, tantangan pas untuk siswa SMA	`tests/run.mjs`, panel statistik
Environment	Papan 7 × 7, dua agen bergiliran. Deterministik, fully observable, diskret, berurutan, zero-sum	`createState()`
Actuators	Lompat ke kotak legal (maks. 8 arah) + hancurkan satu kotak	`applyMove()`, `{ jump, trap }`
Sensors	Posisi kedua knight, status 49 kotak, giliran	`state.cells`, `state.pos`, `state.turn`

Kenapa sifat lingkungan penting? Minimax hanya cocok untuk game *deterministik* (tidak ada dadu), *fully observable* (tidak ada kartu tertutup), dan *zero-sum* (untung saya = rugi lawan). Knight Trap memenuhi ketiganya.

Adversarial search dan game tree

Game tree: akar = posisi sekarang, cabang = langkah, simpul = posisi hasil langkah. **Kedalaman** (depth) = berapa giliran ke depan yang dilihat AI. **Branching factor** = rata-rata jumlah langkah per posisi. Di Knight Trap: sampai 8 lompatan × sampai 12 kandidat jebakan = sampai **96 langkah** per posisi. Dengan depth 4: $96^4 \approx 85$ juta posisi. Itulah alasan kita butuh Alpha-Beta dan pembatasan kandidat.

Minimax dalam satu paragraf

AI adalah **MAX** (ingin skor setinggi mungkin), manusia adalah **MIN** (dianggap selalu memilih langkah terburuk untuk AI). AI menelusuri pohon sampai kedalaman tertentu, menilai posisi di ujung dengan fungsi evaluasi, lalu menaikkan nilai ke atas: di simpul MAX ambil nilai terbesar, di simpul MIN ambil nilai terkecil. Langkah di akar dengan nilai terbesar dipilih.

Alpha-Beta dalam satu paragraf

Alpha = nilai terbaik yang sudah pasti didapat MAX. **Beta** = nilai terbaik yang sudah pasti didapat MIN. Kalau di suatu simpul `beta ≤ alpha`, sisa langkah di simpul itu tidak mungkin mengubah keputusan, jadi dilewati (**cutoff**). Hasilnya **sama persis** dengan Minimax biasa, tapi posisi yang diperiksa jauh lebih sedikit. Ini dibuktikan di pengujian.

BFS dan DFS dalam satu paragraf

BFS (antrian, menyebar per "lapisan") menghitung jumlah lompatan minimum dari knight ke setiap kotak. Kotak yang lebih cepat dicapai knight A adalah **wilayah** A. **DFS** (tumpukan, masuk sedalam mungkin dulu)

melakukan **flood fill**: menandai semua kotak yang bisa dicapai knight. Kalau area kedua knight tidak bersentuhan, papan sudah **terbelah** dan pemenangnya hampir pasti yang areanya lebih luas.

js/ai.js : baris 80–115

```
80  export function evaluate(s, me, ply = 0) {
81    const { cells, pos, turn } = s;
82    const sign = turn === me ? 1 : -1;
83    // The side to move was just eaten, or has no jump: it has lost.
84    if (pos[0] === pos[1] || !hasJump(s, turn)) return -sign * (WIN - ply);
85    // The side to move can eat right now, so it wins on the next ply.
86    if (NEIGHBORS[pos[turn]].includes(pos[turn ^ 1])) return sign * (WIN - ply - 1);
87    const areaA = dfs(cells, pos, 0, markA);
88    const areaB = dfs(cells, pos, 1, markB);
89    let split = true;
90    for (let i = 0; i < SIZE; i++) {
91      if (markA[i] === 1 && markB[i] === 1) {
92        split = false;
93        break;
94      }
95    }
96    let score;
97    if (split) {
98      score = (areaA - areaB) * SPLIT_WEIGHT;
99    } else {
100     bfs(cells, pos, 0, distA);
101     bfs(cells, pos, 1, distB);
102     let terr = 0;
103     for (let i = 0; i < SIZE; i++) {
104       if (!open(cells, pos, i)) continue;
105       const a = distA[i];
106       const b = distB[i];
107       if (a < 0 && b < 0) continue;
108       if (b < 0 || (a >= 0 && a < b)) terr++;
109       else if (a < 0 || b < a) terr--;
110       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
111     }
112     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
113   }
114   return me === 0 ? score : -score;
115 }
```

Inti fungsi evaluasi (skor dari sudut pandang P1, lalu dibalik tandanya untuk P2):

- Cek dulu posisi yang sudah pasti: yang jalan sudah dimakan atau tidak bisa lompat = kalah (`WIN − ply`); yang jalan bisa langsung makan = menang di ply berikutnya (`WIN − ply − 1`).
- Kalau papan sudah terbelah (DFS): `skor = (areaP1 − areaP2) × 100` .
- Kalau belum: `skor = selisih wilayah (BFS) + 0,5 × selisih mobilitas` . Mobilitas = jumlah lompatan legal saat ini.

Angka 100 (`SPLIT_WEIGHT`) dipilih karena skor wilayah paling besar sekitar 53, jadi papan yang sudah terbelah selalu dinilai lebih "pasti" daripada posisi yang masih terbuka.

6 Antarmuka: Bagaimana Papan Digambar

Prinsip: tampilan tidak menyimpan aturan

`board.js` hanya **menggambar**. Ia tidak tahu aturan dan tidak menyimpan state game. `game.js` memberi tahu apa yang harus digambar lewat objek `view`. Pemisahan ini membuat aturan cukup ditulis sekali di `rules.js`.

js/board.js : baris 12–48

```
12    constructor(boardEl, shellEl) {
13      this.el = boardEl;
14      this.shell = shellEl;
15      this.onCell = null;
16      this.onKnight = null;
17      this.cells = [];
18
19      const frag = document.createDocumentFragment();
20      for (let sq = 0; sq < SIZE; sq++) {
21        const { r, c } = rc(sq);
22        const b = document.createElement('button');
23        b.type = 'button';
24        b.className = 'cell';
25        b.dataset.sq = sq;
26        if ((r + c) % 2 === 1) b.classList.add('dark');
27        if (r === N - 1) b.insertAdjacentHTML('beforeend', `<span class="coord f">${'abcdefg'[c]}
</span>`);
28        if (c === 0) b.insertAdjacentHTML('beforeend', `<span class="coord r">${N - r}</span>`);
29        frag.appendChild(b);
30        this.cells.push(b);
31      }
32      this.el.appendChild(frag);
33
34      this.knights = [0, 1].map((side) => {
35        const k = document.createElement('div');
36        k.className = 'knight';
37        k.dataset.side = side;
38        k.innerHTML = knightSvg(side);
39        k.addEventListener('click', () => this.onKnight?.(side));
40        this.el.appendChild(k);
41        return k;
42      });
43
44      this.el.addEventListener('click', (e) => {
45        const b = e.target.closest('.cell');
46        if (b) this.onCell?.(Number(b.dataset.sq));
47      });
48    }
```

- 49 `<button>` dibuat sekali, bukan setiap giliran. Tombol dipakai (bukan `div`) supaya bisa diklik dengan keyboard dan dibaca screen reader.
- Kotak gelap/terang memakai rumus `(r + c) % 2`, sama seperti papan catur.
- Label koordinat (a–g, 1–7) hanya di baris bawah dan kolom kiri.
- Satu listener klik di papan (*event delegation*), bukan 49 listener. `closest('.cell')` mencari kotak yang diklik.

js/board.js : baris 60–93

```
60    render(view) {
61      const { state } = view;
62      const legal = new Set(view.legal || []);
63      const eating = new Set(view.eating || []);
64      const threats = new Set(view.threats || []);
65      for (let sq = 0; sq < SIZE; sq++) {
66        const b = this.cells[sq];
67        const v = state.cells[sq];
68        const occupied = sq === state.pos[0] || sq === state.pos[1];
69        b.classList.toggle('burned', v === BURNED);
70        b.classList.toggle('trap', v === TRAP);
71        const isLegal = legal.has(sq);
72        const isTrappable = view.trappable && v === 0 && !occupied;
73        b.classList.toggle('legal', isLegal);
74        b.classList.toggle('eat', eating.has(sq));
75        b.classList.toggle('trappable', isTrappable);
76        b.classList.toggle('threat', isTrappable && threats.has(sq));
77        b.classList.toggle('last-land', sq === view.lastLand);
78        b.classList.toggle('last-trap', sq === view.lastTrap && v === TRAP);
79        const actionable = isLegal || isTrappable;
80        b.tabIndex = actionable ? 0 : -1;
81        b.setAttribute('aria-disabled', actionable ? 'false' : 'true');
82        const what = v === BURNED ? 'burned' : v === TRAP ? 'trap' : occupied ? 'knight' : 'open';
83        b.setAttribute('aria-label', `${squareName(sq)} ${what}${isLegal ? ', jump here' :
''}${eating.has(sq) ? ', eat' : ''}${isTrappable ? ', place trap' : ''}`);
84      }
85      this.shell.classList.toggle('trap-mode', !!view.trappable);
86      for (const side of [0, 1]) {
87        const k = this.knights[side];
88        this.placeKnight(side, state.pos[side]);
89        k.classList.toggle('is-dead', view.dead === side);
90        k.hidden = view.eaten === side;
91        k.classList.toggle('selectable', view.selectableKnight === side);
92      }
93    }
```

`render()` dipanggil setiap ada perubahan. Ia hanya menyalakan/mematikan **class CSS**: `burned`, `trap`, `legal` (titik merah penanda lompatan, selalu tampil di giliranmu), `eat` (lompatan yang memakan musuh), `trappable`, `threat` (kotak tujuan lawan di fase jebakan; baru terlihat setelah tombol P), `last-land`, `last-trap`. Knight yang dimakan disembunyikan (`hidden`). Semua tampilan visual ditentukan CSS. `aria-label` diisi supaya pembaca layar bisa menyebut "d4 open, jump here".

css/style.css : baris 211–233

```css
.board {
  --gap: 0px;
  position: relative;
  display: grid;
  grid-template-columns: repeat(7, 1fr);
  grid-template-rows: repeat(7, 1fr);
  gap: var(--gap);
  width: 100%;
  height: 100%;
}

.cell {
  position: relative;
  padding: 0;
  overflow: hidden;
  border: 0;
  border-radius: 0;
  background: var(--square-light);
  cursor: default;
}
.cell.dark { background: var(--square-dark); }
.cell.burned { background: var(--burned); }
.cell.trap { background: var(--trap); }
```

Papan memakai **CSS Grid** 7 × 7 (`repeat(7, 1fr)`), jadi ukurannya otomatis mengikuti layar. Warna diambil dari variabel di `:root` (misalnya `--square-light`), sehingga warna cukup diubah di satu tempat. Tampilannya sengaja sederhana: tanpa animasi, efek, atau suara.

Knight dan petunjuk P

js/board.js : baris 50–55

```js
  placeKnight(side, sq) {
    const { r, c } = rc(sq);
    const k = this.knights[side];
    k.style.setProperty('--r', r);
    k.style.setProperty('--c', c);
  }
```

Knight hanya dipindah posisinya dengan CSS `transform`, tanpa animasi. Tombol **P** menyalakan class `hints-on` di bingkai papan; hanya saat class itu ada, kotak `threat` diberi garis putus-putus merah.

Penunjuk giliran dan pesan menang/kalah

js/main.js : baris 139–158

```
139   update(g, text) {
140     const s = g.state;
141     const status = $('#status');
142     const side = g.phase === 'over' ? g.winner : s.turn;
143     status.dataset.side = side;
144     status.classList.toggle('trap-mode', g.phase === 'trap');
145     $('#status-text').textContent = text;
146     $('#undo-jump').hidden = !(g.phase === 'trap' && g.current?.kind === 'human');
147     for (const card of $$('.pcard')) {
148       const sd = Number(card.dataset.side);
149       const p = g.players[sd];
150       $('.pcard-name', card).textContent = p.name;
151       $('.pcard-sub', card).textContent = p.sub;
152       const mob = legalJumps(s, sd).length;
153       $('.pcard-mob b', card).textContent = mob;
154       $('.pcard-mob', card).classList.toggle('danger', g.phase !== 'over' && mob <= 1);
155       card.classList.toggle('is-turn', g.phase !== 'over' && s.turn === sd);
156       card.classList.toggle('is-winner', g.phase === 'over' && g.winner === sd);
157     }
158   },
```

Setiap refresh, kartu pemain menampilkan nama, warna, dan **mobilitas** (jumlah lompatan legal). Kalau tinggal ≤ 1, angkanya merah (`danger`), jadi pemain tahu sedang terancam. Titik di status atas papan berwarna sesuai giliran, garis status menjadi merah saat fase jebakan, dan sisa waktu giliran tampil di ujung kanan status.

js/main.js : baris 201–222

```
201   over(info) {
202     const g = info.game;
203     const me = g.mySide;
204     const winName = g.players[info.winner].name;
205     const loseName = g.players[info.loser].name;
206     let title;
207     let sub;
208     let win = true;
209     const loserSub = t(info.eaten ? 'eatenSub' : 'trappedSub', loseName);
210     if (g.mode === 'local') {
211       title = t('sideWins', winName);
212       sub = loserSub;
213     } else {
214       win = me === info.winner;
215       title = win ? t('youWin') : t('youLose');
216       if (g.mode === 'ai') {
217         if (info.eaten) sub = win ? t('youAteSub') : t('aiAteSub');
218         else sub = win ? t('youWinSub') : t('aiWinSub');
219       } else sub = loserSub;
220     }
221     openGameOver({ title, sub, win, moves: info.moves });
222   },
```

Layar game over menulis keterangan berbeda untuk menang karena makan ("Kamu memakan knight AI") dan menang karena mengurung ("Knight AI terkurung").

Penghitung waktu di layar

js/main.js : baris 162–166

```
162   timer(seconds) {
163     const el = $('#turn-timer');
164     el.textContent = seconds === null ? '' : t('timerLeft', seconds);
165     el.classList.toggle('low', seconds !== null && seconds <= 3);
166   },
```

`game.js` memanggil `ui.timer(detik)` setiap 100 ms selama giliran manusia, dan `ui.timer(null)` untuk menyembunyikannya (giliran AI, game selesai). Detail aturannya ada di bab "Tambahan di Luar Proposal".

7 BFS: Breadth-First Search

Ide

BFS menelusuri graf **per lapisan**: semua kotak berjarak 1 lompatan, lalu semua kotak berjarak 2, dan seterusnya. Karena itu, saat sebuah kotak pertama kali ditemukan, jaraknya pasti yang **terpendek**. Struktur datanya adalah **antrian** (queue, FIFO): yang masuk pertama keluar pertama.

Di sini, **graf** = 49 kotak, dan dua kotak terhubung kalau bisa dicapai dengan satu lompatan knight dan kotak tujuannya masih terbuka.

7	4	3	4	3	4	5	4
6	3	4	3	4	3	4	5
5	2	3	2	3	4	3	4
4	3	2	3	2	3	4	3
3	2	1	4	3	2	3	4
2	3	4	1	2	3	4	3
1	♘	3	2	3	2	3	4
	a	b	c	d	e	f	g

BFS dari knight di a1: angka = jumlah lompatan minimum ke kotak itu (papan kosong).

7	2	3	2	3	2	3	2
6	3	4	1	2	1	4	3
5	2	1	2	3	2	1	2
4	3	2	3	♘	3	2	3
3	2	1	2	3	2	1	2
2	3	4	1	2	1	4	3
1	2	3	2	3	2	3	2
	a	b	c	d	e	f	g

BFS dari knight di d4: angka = jumlah lompatan minimum ke kotak itu (papan kosong).

Perhatikan: dari pojok a1 butuh sampai 5 lompatan untuk mencapai sebagian kotak, sedangkan dari tengah d4 paling jauh 4. Knight di tengah menguasai papan lebih cepat, dan itulah yang diukur oleh wilayah.

Kode BFS

js/ai.js : baris 37–54

```
37  export function bfs(cells, pos, side, dist) {
38    dist.fill(-1);
39    const start = pos[side];
40    dist[start] = 0;
41    let head = 0;
42    let tail = 0;
43    queue[tail++] = start;
44    while (head < tail) {
45      const cur = queue[head++];
46      const d = dist[cur] + 1;
47      for (const n of NEIGHBORS[cur]) {
48        if (dist[n] < 0 && open(cells, pos, n)) {
49          dist[n] = d;
50          queue[tail++] = n;
51        }
52      }
53    }
54  }
```

Baris	Arti
`dist.fill(-1)`	Semua kotak awalnya "belum dicapai" (−1).
`dist[start] = 0`	Kotak knight sendiri berjarak 0.
`queue`, `head`, `tail`	Antrian dibuat dari array biasa: `head` menunjuk yang akan diambil, `tail` tempat memasukkan. Lebih cepat daripada `array.shift()`.
`while (head < tail)`	Selama antrian belum kosong.
`const d = dist[cur] + 1`	Tetangga kotak ini berjarak satu lebih jauh.
`if (dist[n] < 0 && open(...))`	Hanya kotak yang belum dikunjungi dan masih terbuka (tidak terbakar, tidak dijebak, tidak ditempati knight lawan).

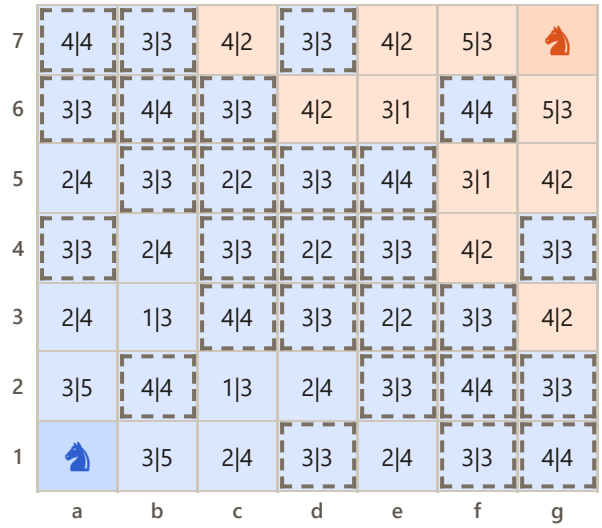
> **Kenapa buffer dibuat sekali di luar fungsi?** `distA`, `distB`, `queue` dibuat sekali sebagai `Int8Array` di atas file. Evaluasi dipanggil puluhan ribu kali per langkah AI; membuat array baru setiap kali akan memperlambat dan membebani garbage collector.

Kompleksitas: setiap kotak masuk antrian paling banyak sekali dan setiap kotak punya maks. 8 tetangga, jadi O(V + E) = maks. 49 + 49 × 8 langkah. Sangat murah.

8 Wilayah (Territory) dan Skor Evaluasi

BFS dijalankan dua kali: dari knight P1 (`distA`) dan dari knight P2 (`distB`). Lalu untuk setiap kotak terbuka:

- Hanya P1 yang bisa mencapai, atau P1 lebih cepat (`a < b`) → wilayah P1 (+1).
- Hanya P2, atau P2 lebih cepat → wilayah P2 (−1).
- **Seri** (`a == b`) → untuk pemain yang jalan berikutnya, karena dia akan sampai duluan.
- Tidak bisa dicapai siapa pun → diabaikan.



Wilayah di posisi awal. Isi kotak = jarak Biru | jarak Oranye. Biru = wilayah P1 (a1), oranye = wilayah P2 (g7), garis putus = seri (dihitung untuk yang jalan berikutnya, P1).

Hasil: P1 37 kotak, P2 10 kotak (termasuk kotak seri untuk P1). Selisih wilayah = 27.

js/ai.js : baris 99–114

```
99    } else {
100     bfs(cells, pos, 0, distA);
101     bfs(cells, pos, 1, distB);
102     let terr = 0;
103     for (let i = 0; i < SIZE; i++) {
104       if (!open(cells, pos, i)) continue;
105       const a = distA[i];
106       const b = distB[i];
107       if (a < 0 && b < 0) continue;
108       if (b < 0 || (a >= 0 && a < b)) terr++;
109       else if (a < 0 || b < a) terr--;
110       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
111     }
112     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
113   }
114   return me === 0 ? score : -score;
```

Lalu `skor = terr + 0,5 × (mobilitas P1 − mobilitas P2)`. Mobilitas diberi bobot kecil (0,5) sebagai pemecah seri: dua posisi dengan wilayah sama, yang lompatannya lebih banyak dianggap sedikit lebih baik. Baris terakhir membalik tanda untuk P2, karena skor selalu dari sudut pandang `me` (pemain AI).

> **Kenapa wilayah masuk akal untuk Knight Trap?** Pemain kalah saat kehabisan lompatan. Kotak yang bisa kamu capai lebih dulu adalah kotak yang kemungkinan besar bisa kamu pakai sebelum lawan membakarnya atau menjebaknya. Semakin besar wilayahmu, semakin banyak "nafas" yang tersisa.

Penyatuan dengan DFS: `evaluate()` menjalankan DFS dulu. Kalau papan sudah terbelah, wilayah tidak lagi relevan (setiap knight hanya bisa memakai areanya sendiri), jadi skor dihitung dari luas area × 100. BFS hanya dijalankan kalau papan belum terbelah. Detail DFS ada di guide Shatrya.

9 Pemain Pembanding (Baseline)

Proposal membandingkan AI dengan dua pemain sederhana. Keduanya ada di bagian bawah `js/ai.js`.

js/ai.js : baris 323–332

```
323 export function randomMove(state, rng = Math.random) {
324   const jumps = legalJumps(state);
325   if (jumps.length === 0) return null;
326   const jump = pickRandom(jumps, rng);
327   if (jump === state.pos[state.turn ^ 1]) return { jump, trap: -1 };
328   const t = cloneState(state);
329   applyJump(t, jump);
330   const opens = openSquares(t);
331   return { jump, trap: opens.length ? pickRandom(opens, rng) : -1 };
332 }
```

Pemain acak: pilih lompatan legal secara acak, lalu jebak kotak terbuka secara acak. `rng` bisa diganti generator dengan seed, supaya pengujian bisa diulang.

js/ai.js : baris 336–373

```
336  export function greedyMove(state, rng = Math.random) {
337    const side = state.turn;
338    const jumps = legalJumps(state);
339    if (jumps.length === 0) return null;
340    if (jumps.includes(state.pos[side ^ 1])) return { jump: state.pos[side ^ 1], trap: -1 };
341    const bestBy = (list, scoreFn) => {
342      let best = -1;
343      let picks = [];
344      for (const x of list) {
345        const v = scoreFn(x);
346        if (v > best) {
347          best = v;
348          picks = [x];
349        } else if (v === best) picks.push(x);
350      }
351      return pickRandom(picks, rng);
352    };
353    const jump = bestBy(jumps, (j) => {
354      const t = cloneState(state);
355      applyJump(t, j);
356      return legalJumps(t, side).length;
357    });
358    const t = cloneState(state);
359    applyJump(t, jump);
360    if (!hasOpenSquare(t)) return { jump, trap: -1 };
361    const oppJumps = legalJumps(t, side ^ 1).filter((sq) => isOpen(t, sq)); // a trap cannot go on our
own knight
362    let trap;
363    if (oppJumps.length) {
364      trap = bestBy(oppJumps, (sq) => {
365        let n = 0;
366        for (const k of NEIGHBORS[sq]) if (isOpen(t, k) && k !== sq) n++;
367        return n;
368      });
369    } else {
370      trap = pickRandom(openSquares(t), rng);
371    }
372    return { jump, trap };
373  }
```

Pemain greedy: kalau bisa makan musuh, langsung makan. Kalau tidak, lompat ke kotak yang memberi lompatan lanjutan terbanyak, lalu jebak kotak tujuan lawan yang paling berharga bagi lawan. Kalau beberapa pilihan sama bagusnya, pilih acak di antaranya (`bestBy`). Greedy hanya melihat satu langkah ke depan, tanpa memikirkan balasan lawan.

10 Pengujian Win Rate dan Waktu Respons

tests/run.mjs : baris 212–263

```
212  // ---------------------------------------------------------------------------
213  console.log(`\n3. Strength (${GAMES} games per level per opponent, AI alternates first/second, 4 s
limit, Alpha-Beta on)`);
214  const timing = {};
215  const winRates = {};
216  {
217    console.log('   level   | depth | vs random | vs greedy');
218    for (const [level, depth] of Object.entries(DIFFICULTY)) {
219      timing[level] = [];
220      winRates[level] = {};
221      for (const [oppName, opp] of [['random', randomMove], ['greedy', greedyMove]]) {
222        let wins = 0;
223        for (let g = 0; g < GAMES; g++) {
224          const rng = mulberry32(BASE_SEED * 7 + depth * 100003 + g * 31 + (oppName === 'greedy' ? 17 :
0));
225          const aiSide = g % 2;
226          const s = createState();
227          let illegal = false;
228          while (winner(s) === null) {
229            let move;
230            if (s.turn === aiSide) {
231              const res = search(s, { depth, prune: true, timeLimitMs: AICFG.timeLimitMs, now:
performance.now.bind(performance) });
232              timing[level].push(res.timeMs);
233              move = res.move;
234            } else {
235              move = opp(s, rng);
236            }
237            if (!validateMove(s, move)) { illegal = true; break; }
238            applyMove(s, move);
239          }
240          check(`${level} vs ${oppName} game ${g}: only legal moves`, !illegal);
241          if (winner(s) === aiSide) wins++;
242        }
243        winRates[level][oppName] = (100 * wins) / GAMES;
244      }
245      console.log(`   ${level.padEnd(6)} | ${pad(depth, 5)} | ${pad(winRates[level].random.toFixed(0) +
'%', 9)} | ${pad(winRates[level].greedy.toFixed(0) + '%', 9)}`);
246      if (depth >= 2) check(`${level}: >= 90% vs random`, winRates[level].random >= 90,
`${winRates[level].random}%`);
247    }
248  }
249
250  // ---------------------------------------------------------------------------
251  console.log('\n4. Response time per AI move (Node, same code as the browser worker)');
252  {
253    console.log('   level   | moves | avg ms | p95 ms | max ms');
254    for (const [level, times] of Object.entries(timing)) {
255      const sorted = [...times].sort((a, b) => a - b);
256      const avg = times.reduce((a, b) => a + b, 0) / times.length;
257      const p95 = sorted[Math.floor(sorted.length * 0.95)];
258      const max = sorted.at(-1);
259      console.log(`   ${level.padEnd(6)} | ${pad(times.length, 5)} | ${pad(avg.toFixed(1), 6)} |
${pad(p95.toFixed(1), 6)} | ${pad(max.toFixed(1), 6)}`);
260      check(`${level}: max < 4000 ms`, max < 4000, `${max.toFixed(0)} ms`);
261      if (level !== 'expert') check(`${level}: max < 1000 ms`, max < 1000, `${max.toFixed(0)} ms`);
```

```
262     }
263 }
```

- **Setiap level** bermain 100 game vs acak dan 100 game vs greedy (`GAMES`; 20 kalau `--quick`).
- **Giliran pertama bergantian** (`aiSide = g % 2`) karena yang jalan pertama punya sedikit keuntungan; supaya adil, AI mendapat setengah game sebagai pemain pertama.
- **Seed per game** dibentuk dari level, nomor game, dan jenis lawan, jadi setiap game bisa diulang persis.
- **Setiap langkah dicek legal** dengan `validateMove`; kalau AI pernah membuat langkah ilegal, tes gagal.
- **Waktu respons**: `res.timeMs` dari setiap langkah AI disimpan, lalu dihitung rata-rata, p95 (diurutkan, ambil posisi 95%), dan maksimum.

11 Hasil Pengujian (Proposal Bab 2.2)

Semua angka di bawah berasal dari `node tests/run.mjs` (run penuh: 100 game per level per lawan). Hasil akhir run: **900 cek lulus, 0 gagal**. Jalankan sendiri sebelum presentasi supaya kamu tahu prosesnya: `node tests/run.mjs --quick` (±30 detik) atau tanpa `--quick` (±5 menit).

1. Kebenaran Alpha-Beta (Tabel 2.2.3, baris 1)

30 posisi uji dibuat dari permainan acak 4–8 giliran dengan seed `20261028 + k` (bisa diulang siapa pun). Alpha-Beta dan Minimax biasa memakai evaluasi, urutan langkah, dan tanpa batas waktu yang sama; bedanya hanya pemangkasan.

Depth	Skor identik	Langkah identik	Node Minimax	Node Alpha-Beta	Hemat	Cutoff	ms Minimax	ms Alpha-Beta
1	30/30	30/30	981	981	0,0%	0	7	5
2	30/30	30/30	34.035	8.888	73,9%	896	120	32
3	30/30	30/30	999.825	54.465	94,6%	3.001	3.042	281

Cara membaca: di depth 3, Alpha-Beta hanya memeriksa sekitar 1 dari 18 posisi yang diperiksa Minimax biasa, ±11× lebih cepat, dan tetap memilih skor dan langkah yang sama di 30 dari 30 posisi. Di depth 1 tidak ada penghematan karena belum ada simpul MIN yang bisa dipangkas (AI hanya melihat langkahnya sendiri).

2. Kekuatan AI / win rate (Tabel 2.2.3, baris 2)

Level	Depth	vs pemain acak	vs pemain greedy
Easy	1	99%	89%
Medium	2	99%	100%
Hard	3	100%	79%
Expert	4	100%	92%

Target proposal "Medium ke atas menang minimal 90% melawan pemain acak" **tercapai** (99–100%). Melawan greedy, win rate **tidak naik seiring level**: Medium 100%, tetapi Hard hanya 79%, di bawah Easy (89%). Jadi target "win rate naik seiring level" **tidak tercapai** dengan aturan makan yang sekarang. Penyebabnya belum diteliti dengan uji khusus; yang bisa disebut sebagai dugaan:

- **Kedalaman ganjil dan genap.** Depth 1 dan 3 berhenti setelah langkah AI sendiri, depth 2 dan 4 setelah balasan lawan. Dengan aturan makan, hasilnya bisa berbeda cukup jauh antara ganjil dan genap.
- **Greedy selalu makan kalau bisa**, dan hanya pemain Oranye yang bisa makan. Karena AI bergantian jadi Biru dan Oranye, di separuh game AI hanya bisa menang dengan mengurung.
- Fungsi evaluasi BFS/DFS adalah perkiraan, dan pembatasan 12 kandidat jebakan bisa membuang langkah yang ternyata bagus.
- Dengan 100 game, selisih beberapa poin persen masih bisa karena variasi acak, tetapi selisih Medium–Hard (21 poin) terlalu besar untuk dijelaskan hanya dengan itu.

Kalau ditanya, sampaikan angka ini apa adanya sebagai temuan pengujian, dan jangan menyebut satu penyebab sebagai yang pasti.

3. Waktu respons (Tabel 2.2.3, baris 3)

Level	Langkah diukur	Rata-rata	p95	Maksimum
Easy	957	0,2 ms	0,3 ms	10,1 ms
Medium	826	1,2 ms	4,1 ms	8,0 ms
Hard	795	17,2 ms	50,9 ms	92,2 ms
Expert	842	148,2 ms	443,4 ms	696,1 ms

Target "semua level di bawah 4 detik, Easy–Hard di bawah 1 detik" **tercapai jauh**. Angka ini diukur di Node.js dengan kode yang sama dengan Web Worker di browser; di laptop yang lebih lambat angkanya bisa lebih besar, tetapi masih jauh dari 4 detik. *p95* artinya 95% langkah selesai di bawah angka itu.

4. Hal yang belum dibuktikan oleh tes otomatis

- **Playtest** (±10 siswa, kuesioner skala 1–5) adalah uji manusia; hasilnya harus dicatat sendiri oleh tim. Jangan sebut angka playtest yang tidak benar-benar dilakukan.
- Waktu respons di browser bisa dicek langsung di panel **Statistik AI** saat bermain (kolom Waktu).

tests/run.mjs : baris 16–22

```
16  function check(name, cond, detail = '') {
17    if (cond) passes++;
18    else {
19      failures++;
20      console.log(`  FAIL ${name}${detail ? ' :: ' + detail : ''}`);
21    }
22  }
```

12 Tambahan di Luar Proposal: Makan dan Waktu Giliran

Dua aturan ini **tidak ada di proposal**, tetapi ada di game, jadi bisa ditanya saat demo. Bukan bagian dari empat algoritma AI. Versi sekarang juga sudah tidak punya poin, leaderboard, suara, atau animasi; tampilannya dibuat sederhana.

Makan

Aturan (d) di proposal melarang mendarat di kotak lawan. Di kode, mendarat **tepat** di kotak knight musuh justru **memakan** musuh, seperti di catur: game langsung selesai, tidak ada jebakan, dan yang makan menang. Bersebelahan dengan musuh tidak berarti apa-apa; game tetap jalan.

Tempat	Yang dilakukan
`legalJumps()`, `canMove()`	Kotak musuh dihitung sebagai lompatan legal (cek `cells[n] === EMPTY`, bukan `isOpen`).
`isOpen()`	Tetap melarang jebakan di kotak yang ada knight-nya.
`validateMove()`	Langkah makan sah hanya dengan `trap = -1`.
`winner()`	Dua knight di satu kotak berarti yang sedang jalan baru saja dimakan, jadi lawannya menang.
`Game.handleCell()`	Klik di kotak musuh langsung mengakhiri giliran tanpa fase jebakan.
`ai.js`	`generateMoves()` membuat langkah makan sebagai `pack(j, -1)`; `orderMoves()` memeriksanya paling dulu; `ab()` dan `evaluate()` menilai "dimakan" sebagai kalah dan "bisa makan" sebagai menang di ply berikutnya.

tests/run.mjs : baris 85–96

```
85    // Eating: landing exactly on the enemy's square wins at once, with no trap.
86    const eatBlue = createState({ p1: sq('a1'), p2: sq('c2'), first: 0 });
87    check('blue eats: no trap is valid', validateMove(eatBlue, { jump: sq('c2'), trap: -1 }));
88    check('blue eats: a real trap is rejected', !validateMove(eatBlue, { jump: sq('c2'), trap: sq('a7')
}));
89    applyMove(eatBlue, { jump: sq('c2'), trap: -1 });
90    check('blue eats: blue wins', winner(eatBlue) === 0);
91    check('blue eats: both knights share the square', eatBlue.pos[0] === sq('c2') && eatBlue.pos[1] ===
sq('c2'));
92
93    const eatOrange = createState({ p1: sq('e6'), p2: sq('g7'), first: 1 });
94    check('orange eats: no trap is valid', validateMove(eatOrange, { jump: sq('e6'), trap: -1 }));
95    applyMove(eatOrange, { jump: sq('e6'), trap: -1 });
96    check('orange eats: orange wins', winner(eatOrange) === 1);
```

> **Kenapa hanya Oranye yang bisa makan?** Setiap lompatan knight selalu pindah warna kotak (terang ↔ gelap). a1 dan g7 berwarna sama. Biru selalu jalan duluan, jadi pada giliran Biru kedua knight sudah melompat sama banyak dan berdiri di warna yang sama; lompatan Biru pasti mendarat di warna lain, jadi tidak mungkin kena Oranye. Pada giliran Oranye, Biru sudah melompat sekali lebih banyak, jadi warnanya berbeda dan lompatan Oranye bisa mendarat di kotak Biru. Akibatnya Biru hanya bisa menang dengan mengurung. Mengubah posisi awal hanya menukar sisi mana yang bisa makan.

Waktu giliran

Satu giliran manusia maksimal `TURN_SECONDS` (15 detik, di `config.js`) untuk lompat **dan** jebak sekaligus. Sisa waktu tampil di samping status giliran dan menjadi merah saat tinggal 3 detik.

`js/game.js : baris 165–174`

```
165    tickClock() {
166      const left = this.deadline - performance.now();
167      this.ui.timer(Math.max(0, Math.ceil(left / 1000)));
168      if (this.current.kind === 'human') {
169        if (left <= 0) this.timeUp();
170      } else if (this.role === 'host' && left <= -TURN_GRACE_MS) {
171        this.stopClock();
172        this.netQueue = this.netQueue.then(() => this.playForQuietGuest()).catch((err) =>
console.error(err));
173      }
174    }
```

`js/game.js : baris 178–191`

```
178    timeUp() {
179      this.stopClock();
180      this.ui.timeUp();
181      const s = this.state;
182      if (this.phase === 'jump') {
183        const { jump, trap } = randomMove(s);
184        this.handleCell(jump);
185        if (this.phase === 'trap') this.handleCell(trap);
186      } else if (this.phase === 'trap') {
187        const open = [];
188        for (let sq = 0; sq < SIZE; sq++) if (isOpen(s, sq)) open.push(sq);
189        this.handleCell(open[Math.floor(Math.random() * open.length)]);
190      }
191    }
```

- **Waktu habis sebelum lompat:** `randomMove()` memilih langkah acak yang lengkap (lompatan + jebakan). **Sudah lompat:** lompatanmu tetap, hanya jebakannya yang dipilih acak.
- Langkah acak dimasukkan lewat `handleCell()`, sama seperti klik, jadi aturan dan pengiriman online tidak ditulis dua kali.
- **Batal lompat tidak mengulang waktu:** `deadline` tidak berubah.
- **AI tidak memakai penghitung ini.** AI punya batas 4 detik sendiri (bagian Louis).
- **Online:** perangkat masing-masing pemain bertindak saat waktunya habis. Kalau tamu diam lebih dari `TURN_GRACE_MS` (1,5 detik) setelah batas, host memainkan langkah acak untuknya.
- Bagian 6 di `tests/run.mjs` menguji semua hal di atas dengan batas waktu yang dibuat pendek.

Petunjuk di papan

Kotak yang bisa kamu tuju selalu ditandai titik merah. Kotak yang bisa dituju musuh (di fase jebakan) disembunyikan, dan baru tampil bergaris putus-putus setelah tombol **P** ditekan. Setiap game baru mulai dengan petunjuk musuh tersembunyi.

13 Latihan

Latihan 1: BFS dengan tangan

Knight di b1 (papan kosong). Sebutkan semua kotak berjarak 1. Berapa jarak ke b2?

Jawaban: jarak 1: a3, c3, d2. b2 butuh 3 lompatan (contoh: b1 → d2 → c4 → b2). Kotak yang bersebelahan langsung justru jauh untuk knight.

Latihan 2: Aturan seri

Sebuah kotak berjarak 2 dari P1 dan 2 dari P2, dan sekarang giliran P2. Kotak itu masuk wilayah siapa? Mengapa?

Jawaban: P2, karena P2 yang jalan berikutnya dan akan mencapainya lebih dulu (`terr += turn === 0 ? 1 : -1`).

Latihan 3: Coba di kode

1. Di `evaluate()`, ubah bobot mobilitas 0,5 menjadi 0, jalankan `node tests/run.mjs --quick`, bandingkan win rate vs greedy. (Kembalikan setelahnya.)
2. Di `css/style.css`, ubah `--square-light` dan `--square-dark` lalu buka game. Semua kotak ikut berubah karena memakai variabel.

14 Pertanyaan Presentasi yang Mungkin Muncul

T: Kenapa BFS, bukan DFS, untuk menghitung jarak?

J: BFS menelusuri per lapisan, jadi saat sebuah kotak pertama kali ditemukan, jaraknya pasti terpendek. DFS tidak menjamin jarak terpendek.

T: Apa itu wilayah dan kenapa dipakai di evaluasi?

J: Kotak yang bisa dicapai knight kita lebih cepat dari knight lawan. Karena kalah = kehabisan lompatan, punya lebih banyak kotak "milik sendiri" berarti bisa bertahan lebih lama.

T: Kenapa ada 0,5 × mobilitas?

J: Sebagai pemecah seri. Bobotnya kecil supaya wilayah tetap faktor utama, tetapi posisi dengan lompatan lebih banyak sedikit diutamakan.

T: Bagaimana win rate diukur supaya adil?

J: 100 game per level per lawan, AI bergantian jalan pertama, seed tercatat, batas waktu 4 detik aktif, dan setiap langkah dicek legal.

T: Kenapa halaman tidak macet saat AI berpikir?

J: AI berjalan di Web Worker (thread lain). Klik, tombol, dan tampilan ada di thread utama, jadi keduanya tidak saling menunggu.

T: Bagaimana pemain tahu lompatan mana yang legal?

J: `game.view()` mengisi `legal` dengan `legalJumps()`, lalu `render()` memberi class `legal` yang menampilkan titik di kotak itu. Lompatan yang memakan musuh juga diberi class `eat`. Di fase jebakan, kotak tujuan lawan diberi class `threat`, tetapi garisnya baru terlihat setelah pemain menekan tombol P.

15 Checklist Sebelum Presentasi

- ☐ Bisa menghitung BFS knight dengan tangan di papan kecil.
- ☐ Bisa menjelaskan papan wilayah posisi awal dan aturan seri.
- ☐ Bisa menunjuk `render()` dan menjelaskan class CSS yang dinyalakan.
- ☐ Hafal hasil win rate (99–100% vs acak; 89/100/79/92% vs greedy) dan waktu (Expert maks. 696 ms).
- ☐ Bisa menjelaskan dengan jujur kenapa win rate vs greedy tidak naik seiring level (Hard paling rendah).
- ☐ Bisa menjelaskan alur satu giliran (Bab 4) tanpa melihat catatan.